

Primality Testing: Theory, Implementation, and Experimental Comparison

An Advanced Dive into Miller-Rabin, Arbitrary Precision, and Benchmarks

Group No. 230272

CS648 Project Presentation

April 5, 2026

Table of Contents

1. The Motivation & Problem Statement
2. Mathematical Build-up (The Proofs)
3. The Miller-Rabin Test
4. Error Bounds for Composites
5. Software Engineering and Implementation Architecture
6. Empirical Benchmarks and Practical Inferences
7. Conclusion

The Primality Problem

Primality testing asks a deceptively simple question: Is a given positive integer n prime or composite?

The Primality Problem

Primality testing asks a deceptively simple question: Is a given positive integer n prime or composite?

Why do we need computationally fast testing?

The Primality Problem

Primality testing asks a deceptively simple question: Is a given positive integer n prime or composite?

Why do we need computationally fast testing?

- **Public-Key Cryptography (RSA):** Requires the generation of extremely large prime keys (e.g., 2048 to 4096 bits).

The Primality Problem

Primality testing asks a deceptively simple question: Is a given positive integer n prime or composite?

Why do we need computationally fast testing?

- **Public-Key Cryptography (RSA):** Requires the generation of extremely large prime keys (e.g., 2048 to 4096 bits).
- **Generating Primes:** Generating a prime practically means generating random odd numbers and testing them until one passes.

The Primality Problem

Primality testing asks a deceptively simple question: Is a given positive integer n prime or composite?

Why do we need computationally fast testing?

- **Public-Key Cryptography (RSA):** Requires the generation of extremely large prime keys (e.g., 2048 to 4096 bits).
- **Generating Primes:** Generating a prime practically means generating random odd numbers and testing them until one passes.
- **The Catch:** Standard exact factoring tests scale exponentially. Trying to divide a 4096-bit number by all odd integers up to its square root would take longer than the lifespan of the universe.

The Primality Problem

Primality testing asks a deceptively simple question: Is a given positive integer n prime or composite?

Why do we need computationally fast testing?

- **Public-Key Cryptography (RSA):** Requires the generation of extremely large prime keys (e.g., 2048 to 4096 bits).
- **Generating Primes:** Generating a prime practically means generating random odd numbers and testing them until one passes.
- **The Catch:** Standard exact factoring tests scale exponentially. Trying to divide a 4096-bit number by all odd integers up to its square root would take longer than the lifespan of the universe.

The Engineering Dilemma

We cannot afford mathematically "perfect but slow" deterministic tests. We need an algorithm resolving 4096-bit computations in milliseconds.

Baseline Alternatives

Before covering the optimal solution, let's look at the benchmarks.

Baseline Alternatives

Before covering the optimal solution, let's look at the benchmarks.

The Square-Root Method

- Tests divisibility by integers up to $\sqrt{n}$.
- Exact, but strictly limited by an $\mathcal{O}(\sqrt{n})$ runtime. Hits a wall around 50 bits.

Baseline Alternatives

Before covering the optimal solution, let's look at the benchmarks.

The Square-Root Method

- Tests divisibility by integers up to $\sqrt{n}$.
- Exact, but strictly limited by an $\mathcal{O}(\sqrt{n})$ runtime. Hits a wall around 50 bits.

The AKS Primality Test

- Revolutionized theory by proving **PRIMES** $\in$ **P**.
- Deterministic polynomial time.
- However, the overhead constant is gigantic. In our implementation, testing a 60-bit prime with AKS already stalls execution significantly.
- The AKS code in this project was written by ChatGPT.

Reference: M. Agrawal, N. Kayal, and N. Saxena, *PRIMES is in P*, Annals of Mathematics, 160(2), 781–793, 2004.

Fermat's Little Theorem

The classical starting point for primality testing is Fermat's Little Theorem.

Fermat's Little Theorem

The classical starting point for primality testing is Fermat's Little Theorem.

Theorem

If p is prime and $\gcd(a, p) = 1$, then:

$$a^{p-1} \equiv 1 \pmod{p}$$

Fermat's Little Theorem

The classical starting point for primality testing is Fermat's Little Theorem.

Theorem

If p is prime and $\gcd(a, p) = 1$, then:

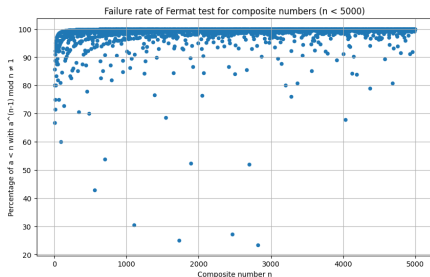
$$a^{p-1} \equiv 1 \pmod{p}$$

The Carmichael Trap (The Weakness)

Unfortunately, reversing the theorem is imperfect. There are composite numbers n that satisfy $a^{n-1} \equiv 1 \pmod{n}$ for *all* bases a coprime to n .

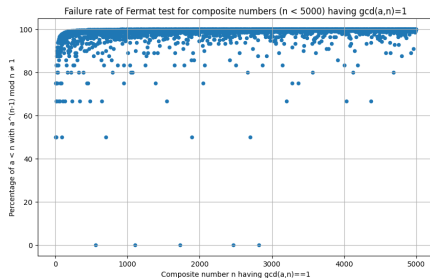
These are called **Carmichael numbers** (e.g., $561 = 3 \times 11 \times 17$).

Fermat Experiments on Composite Numbers



All bases $a < n$: most composite numbers fail Fermat for a large fraction of bases.

Key observation: For non-Carmichael composites, the coprime-base failure rate stays at or above 50%, which matches the half-witness result used to motivate a stronger test.



Only coprime bases: Carmichael numbers appear as low-failure outliers.

Building a Stronger Test: Step 1 (GCD Failures)

Question: Which bases a can trick the Fermat test when n is composite?

Building a Stronger Test: Step 1 (GCD Failures)

Question: Which bases a can trick the Fermat test when n is composite?

Lemma

If $1 < a < n$ and $\gcd(a, n) > 1$, then $a^{n-1} \not\equiv 1 \pmod{n}$.

Building a Stronger Test: Step 1 (GCD Failures)

Question: Which bases a can trick the Fermat test when n is composite?

Lemma

If $1 < a < n$ and $\gcd(a, n) > 1$, then $a^{n-1} \not\equiv 1 \pmod{n}$.

Proof.

- Let $g = \gcd(a, n) > 1$.



Building a Stronger Test: Step 1 (GCD Failures)

Question: Which bases a can trick the Fermat test when n is composite?

Lemma

If $1 < a < n$ and $\gcd(a, n) > 1$, then $a^{n-1} \not\equiv 1 \pmod{n}$.

Proof.

- Let $g = \gcd(a, n) > 1$.
- Assume for contradiction $a^{n-1} \equiv 1 \pmod{n}$. This implies $n \mid (a^{n-1} - 1)$.



Building a Stronger Test: Step 1 (GCD Failures)

Question: Which bases a can trick the Fermat test when n is composite?

Lemma

If $1 < a < n$ and $\gcd(a, n) > 1$, then $a^{n-1} \not\equiv 1 \pmod{n}$.

Proof.

- Let $g = \gcd(a, n) > 1$.
- Assume for contradiction $a^{n-1} \equiv 1 \pmod{n}$. This implies $n \mid (a^{n-1} - 1)$.
- Since $g \mid n$, we have $g \mid (a^{n-1} - 1)$.



Building a Stronger Test: Step 1 (GCD Failures)

Question: Which bases a can trick the Fermat test when n is composite?

Lemma

If $1 < a < n$ and $\gcd(a, n) > 1$, then $a^{n-1} \not\equiv 1 \pmod{n}$.

Proof.

- Let $g = \gcd(a, n) > 1$.
- Assume for contradiction $a^{n-1} \equiv 1 \pmod{n}$. This implies $n \mid (a^{n-1} - 1)$.
- Since $g \mid n$, we have $g \mid (a^{n-1} - 1)$.
- Since $g \mid a$, we have $g \mid a^{n-1}$.



Building a Stronger Test: Step 1 (GCD Failures)

Question: Which bases a can trick the Fermat test when n is composite?

Lemma

If $1 < a < n$ and $\gcd(a, n) > 1$, then $a^{n-1} \not\equiv 1 \pmod{n}$.

Proof.

- Let $g = \gcd(a, n) > 1$.
- Assume for contradiction $a^{n-1} \equiv 1 \pmod{n}$. This implies $n \mid (a^{n-1} - 1)$.
- Since $g \mid n$, we have $g \mid (a^{n-1} - 1)$.
- Since $g \mid a$, we have $g \mid a^{n-1}$.
- Thus g divides their difference: $a^{n-1} - (a^{n-1} - 1) = 1$.



Building a Stronger Test: Step 1 (GCD Failures)

Question: Which bases a can trick the Fermat test when n is composite?

Lemma

If $1 < a < n$ and $\gcd(a, n) > 1$, then $a^{n-1} \not\equiv 1 \pmod{n}$.

Proof.

- Let $g = \gcd(a, n) > 1$.
- Assume for contradiction $a^{n-1} \equiv 1 \pmod{n}$. This implies $n \mid (a^{n-1} - 1)$.
- Since $g \mid n$, we have $g \mid (a^{n-1} - 1)$.
- Since $g \mid a$, we have $g \mid a^{n-1}$.
- Thus g divides their difference: $a^{n-1} - (a^{n-1} - 1) = 1$.
- A contradiction, as no $g > 1$ can divide 1.



Conclusion: Only bases coprime to n can possibly fool Fermat.

Building a Stronger Test: Step 2 (Mappings)

To establish probability bounds, we look at mappings over the group.

Building a Stronger Test: Step 2 (Mappings)

To establish probability bounds, we look at mappings over the group.

Lemma

Let a and p be coprime. For integers $1 \leq i, j < p$ and $i \neq j$, we have:

$$(a \cdot i) \bmod p \neq (a \cdot j) \bmod p$$

Building a Stronger Test: Step 2 (Mappings)

To establish probability bounds, we look at mappings over the group.

Lemma

Let a and p be coprime. For integers $1 \leq i, j < p$ and $i \neq j$, we have:

$$(a \cdot i) \bmod p \neq (a \cdot j) \bmod p$$

Proof.

- Assume $(a \cdot i) \bmod p = (a \cdot j) \bmod p$.



Building a Stronger Test: Step 2 (Mappings)

To establish probability bounds, we look at mappings over the group.

Lemma

Let a and p be coprime. For integers $1 \leq i, j < p$ and $i \neq j$, we have:

$$(a \cdot i) \bmod p \neq (a \cdot j) \bmod p$$

Proof.

- Assume $(a \cdot i) \bmod p = (a \cdot j) \bmod p$.
- Then $a \cdot (i - j) \equiv 0 \pmod{p}$.



Building a Stronger Test: Step 2 (Mappings)

To establish probability bounds, we look at mappings over the group.

Lemma

Let a and p be coprime. For integers $1 \leq i, j < p$ and $i \neq j$, we have:

$$(a \cdot i) \bmod p \neq (a \cdot j) \bmod p$$

Proof.

- Assume $(a \cdot i) \bmod p = (a \cdot j) \bmod p$.
- Then $a \cdot (i - j) \equiv 0 \pmod{p}$.
- Since $\gcd(a, p) = 1$, by Bézout's identity $ax + py = 1$.



Building a Stronger Test: Step 2 (Mappings)

To establish probability bounds, we look at mappings over the group.

Lemma

Let a and p be coprime. For integers $1 \leq i, j < p$ and $i \neq j$, we have:

$$(a \cdot i) \bmod p \neq (a \cdot j) \bmod p$$

Proof.

- Assume $(a \cdot i) \bmod p = (a \cdot j) \bmod p$.
- Then $a \cdot (i - j) \equiv 0 \pmod{p}$.
- Since $\gcd(a, p) = 1$, by Bézout's identity $ax + py = 1$.
- Multiplying both sides by x implies that $(i - j) \equiv 0 \pmod{p}$.



Building a Stronger Test: Step 2 (Mappings)

To establish probability bounds, we look at mappings over the group.

Lemma

Let a and p be coprime. For integers $1 \leq i, j < p$ and $i \neq j$, we have:

$$(a \cdot i) \bmod p \neq (a \cdot j) \bmod p$$

Proof.

- Assume $(a \cdot i) \bmod p = (a \cdot j) \bmod p$.
- Then $a \cdot (i - j) \equiv 0 \pmod{p}$.
- Since $\gcd(a, p) = 1$, by Bézout's identity $ax + py = 1$.
- Multiplying both sides by x implies that $(i - j) \equiv 0 \pmod{p}$.
- Since $1 \leq i, j < p$, this contradicts $i \neq j$.



Conclusion: Modular multiplication by a coprime element acts as a pure bijection.

Building a Stronger Test: Step 3 (Half Witnesses)

Proposition

*If there is **at least one** coprime witness x_j that fails Fermat's check modulo a , then **at least half** of all coprime elements fail it.*

Building a Stronger Test: Step 3 (Half Witnesses)

Proposition

If there is **at least one** coprime witness x_j that fails Fermat's check modulo a , then **at least half** of all coprime elements fail it.

Proof Sketch.

- Let S be the set of elements coprime to a . Assume exactly one $x_j \in S$ is a witness: $x_j^{a-1} \not\equiv 1 \pmod{a}$.



Building a Stronger Test: Step 3 (Half Witnesses)

Proposition

*If there is **at least one** coprime witness x_j that fails Fermat's check modulo a , then **at least half** of all coprime elements fail it.*

Proof Sketch.

- Let S be the set of elements coprime to a . Assume exactly one $x_j \in S$ is a witness: $x_j^{a-1} \not\equiv 1 \pmod{a}$.
- Suppose for contradiction more than half the elements in S are "liars" (they evaluate to 1). Call this set S' .



Building a Stronger Test: Step 3 (Half Witnesses)

Proposition

If there is **at least one** coprime witness x_j that fails Fermat's check modulo a , then **at least half** of all coprime elements fail it.

Proof Sketch.

- Let S be the set of elements coprime to a . Assume exactly one $x_j \in S$ is a witness: $x_j^{a-1} \not\equiv 1 \pmod{a}$.
- Suppose for contradiction more than half the elements in S are "liars" (they evaluate to 1). Call this set S' .
- Construct $S'' = \{(x_j \cdot x_i) \bmod a \mid x_i \in S'\}$.



Building a Stronger Test: Step 3 (Half Witnesses)

Proposition

If there is **at least one** coprime witness x_j that fails Fermat's check modulo a , then **at least half** of all coprime elements fail it.

Proof Sketch.

- Let S be the set of elements coprime to a . Assume exactly one $x_j \in S$ is a witness: $x_j^{a-1} \not\equiv 1 \pmod{a}$.
- Suppose for contradiction more than half the elements in S are "liars" (they evaluate to 1). Call this set S' .
- Construct $S'' = \{(x_j \cdot x_i) \bmod a \mid x_i \in S'\}$.
- Every element in S'' gives: $(x_j \cdot x_i)^{a-1} \equiv x_j^{a-1} \cdot 1 \not\equiv 1 \pmod{a}$.



Building a Stronger Test: Step 3 (Half Witnesses)

Proposition

If there is **at least one** coprime witness x_j that fails Fermat's check modulo a , then **at least half** of all coprime elements fail it.

Proof Sketch.

- Let S be the set of elements coprime to a . Assume exactly one $x_j \in S$ is a witness: $x_j^{a-1} \not\equiv 1 \pmod{a}$.
- Suppose for contradiction more than half the elements in S are "liars" (they evaluate to 1). Call this set S' .
- Construct $S'' = \{(x_j \cdot x_i) \bmod a \mid x_i \in S'\}$.
- Every element in S'' gives: $(x_j \cdot x_i)^{a-1} \equiv x_j^{a-1} \cdot 1 \not\equiv 1 \pmod{a}$.
- Thus, S'' is a set of witnesses exactly the same size as S' .



Building a Stronger Test: Step 3 (Half Witnesses)

Proposition

If there is **at least one** coprime witness x_j that fails Fermat's check modulo a , then **at least half** of all coprime elements fail it.

Proof Sketch.

- Let S be the set of elements coprime to a . Assume exactly one $x_j \in S$ is a witness: $x_j^{a-1} \not\equiv 1 \pmod{a}$.
- Suppose for contradiction more than half the elements in S are "liars" (they evaluate to 1). Call this set S' .
- Construct $S'' = \{(x_j \cdot x_i) \bmod a \mid x_i \in S'\}$.
- Every element in S'' gives: $(x_j \cdot x_i)^{a-1} \equiv x_j^{a-1} \cdot 1 \not\equiv 1 \pmod{a}$.
- Thus, S'' is a set of witnesses exactly the same size as S' .
- This forces $|S| = |S'| + |S''| > |S|$, a mathematical impossibility.



The Miller-Rabin Approach

Miller-Rabin circumvents Carmichael numbers by tracking the **root sequence** leading to 1.

The Miller-Rabin Approach

Miller-Rabin circumvents Carmichael numbers by tracking the **root sequence** leading to 1.

1. Factor Out Powers of 2 For an odd integer $n > 2$, write:

$$n - 1 = 2^s d \quad (\text{where } d \text{ is odd})$$

The Miller-Rabin Approach

Miller-Rabin circumvents Carmichael numbers by tracking the **root sequence** leading to 1.

1. Factor Out Powers of 2 For an odd integer $n > 2$, write:

$$n - 1 = 2^s d \quad (\text{where } d \text{ is odd})$$

2. Squaring Sequence For a chosen base a , define:

$$x_0 = a^d \pmod{n}$$

$$x_1 = x_0^2, x_2 = x_1^2, \dots, x_s = x_{s-1}^2 \pmod{n}$$

The Miller-Rabin Approach

Miller-Rabin circumvents Carmichael numbers by tracking the **root sequence** leading to 1.

1. Factor Out Powers of 2 For an odd integer $n > 2$, write:

$$n - 1 = 2^s d \quad (\text{where } d \text{ is odd})$$

2. Squaring Sequence For a chosen base a , define:

$$x_0 = a^d \pmod{n}$$

$$x_1 = x_0^2, x_2 = x_1^2, \dots, x_s = x_{s-1}^2 \pmod{n}$$

3. Acceptance Rule If n is prime, the sequence must either:

- Start as 1 ($x_0 \equiv 1 \pmod{n}$)
- Encounter -1 right before it hits 1 ($x_r \equiv -1 \pmod{n}$)

Why Do Primes Never Fail?

Lemma

If p is an odd prime, and $x^2 \equiv 1 \pmod{p}$, then $x \equiv \pm 1 \pmod{p}$.

Why Do Primes Never Fail?

Lemma

If p is an odd prime, and $x^2 \equiv 1 \pmod{p}$, then $x \equiv \pm 1 \pmod{p}$.

Proof.

$x^2 - 1 \equiv 0 \implies (x - 1)(x + 1) \equiv 0 \pmod{p}$. Since p is prime, $\mathbb{Z}/p\mathbb{Z}$ has no zero divisors, so $x \equiv 1$ or $x \equiv -1$. □

Why Do Primes Never Fail?

Lemma

If p is an odd prime, and $x^2 \equiv 1 \pmod{p}$, then $x \equiv \pm 1 \pmod{p}$.

Proof.

$x^2 - 1 \equiv 0 \implies (x - 1)(x + 1) \equiv 0 \pmod{p}$. Since p is prime, $\mathbb{Z}/p\mathbb{Z}$ has no zero divisors, so $x \equiv 1$ or $x \equiv -1$. □

Theorem

If n is an odd prime, the Miller-Rabin test never falsely rejects it.

Why Do Primes Never Fail?

Lemma

If p is an odd prime, and $x^2 \equiv 1 \pmod{p}$, then $x \equiv \pm 1 \pmod{p}$.

Proof.

$x^2 - 1 \equiv 0 \implies (x-1)(x+1) \equiv 0 \pmod{p}$. Since p is prime, $\mathbb{Z}/p\mathbb{Z}$ has no zero divisors, so $x \equiv 1$ or $x \equiv -1$. □

Theorem

If n is an odd prime, the Miller-Rabin test never falsely rejects it.

Proof Summary.

Fermat dictates the sequence ends at 1 ($x_s = 1$). If $x_0 \not\equiv 1$, there must be a **first** index j where $x_j \equiv 1$. Therefore, $x_{j-1}^2 \equiv 1$. By the lemma, x_{j-1} must be -1 . Thus, the sequence perfectly satisfies the Miller-Rabin accept condition. □

Bounding the Strong Liars

Composite integers that manage to pass the test are called **strong liars**. How small is this set probabilistically?

Bounding the Strong Liars

Composite integers that manage to pass the test are called **strong liars**. How small is this set probabilistically?

The Group Theoretic Verification: We do not present the proof in this talk. We only use the final group-theoretic result for Miller-Rabin.

Bounding the Strong Liars

Composite integers that manage to pass the test are called **strong liars**. How small is this set probabilistically?

The Group Theoretic Verification: We do not present the proof in this talk. We only use the final group-theoretic result for Miller-Rabin.

- For any odd composite number n , the bases that falsely make Miller-Rabin accept are only a limited fraction of all valid bases.
- The final bound is that at most one fourth of the possible bases are strong liars.

Bounding the Strong Liars

Composite integers that manage to pass the test are called **strong liars**. How small is this set probabilistically?

The Group Theoretic Verification: We do not present the proof in this talk. We only use the final group-theoretic result for Miller-Rabin.

- For any odd composite number n , the bases that falsely make Miller-Rabin accept are only a limited fraction of all valid bases.
- The final bound is that at most one fourth of the possible bases are strong liars.

Final Result

If n is an odd composite, the set of strong liars satisfies:

$$|L(n)| \leq \frac{\varphi(n)}{4}$$

Probabilistic Absolute Certainty

Because $|L(n)| \leq \frac{\varphi(n)}{4}$:

Probabilistic Absolute Certainty

Because $|L(n)| \leq \frac{\varphi(n)}{4}$:

- For an odd composite, a random base a passes with a highest probability of $\frac{1}{4}$.
- By testing k independent bases, the probability of false acceptance becomes $\leq (\frac{1}{4})^k$.

Probabilistic Absolute Certainty

Because $|L(n)| \leq \frac{\varphi(n)}{4}$:

- For an odd composite, a random base a passes with a highest probability of $\frac{1}{4}$.
- By testing k independent bases, the probability of false acceptance becomes $\leq (\frac{1}{4})^k$.

Practical Implementation

If $k = 40$ random bases are tested independently, the chance of a composite slipping through is $\leq 4^{-40} \approx 8.27 \times 10^{-25}$. It is computationally flawless for any realistic threshold.

C++ Implementation Workflow

Theory dictates functionality, but engineering provides execution.

C++ Implementation Workflow

Theory dictates functionality, but engineering provides execution.

1. The `big_int` Class Context

- Normal 64-bit integer registers overflow instantly for cryptographic parameters.
- Our custom C++ class stores numbers in arrays mapped in 10^9 block intervals.
- Overloads logic for addition, subtraction, arithmetic shifting, and most crucially, modular division.

C++ Implementation Workflow

Theory dictates functionality, but engineering provides execution.

1. The `big_int` Class Context

- Normal 64-bit integer registers overflow instantly for cryptographic parameters.
- Our custom C++ class stores numbers in arrays mapped in 10^9 block intervals.
- Overloads logic for addition, subtraction, arithmetic shifting, and most crucially, modular division.

2. Specialized `fast_mod`

- In $a^b \pmod n$, repeated squaring causes enormous overhead.
- We implemented a dedicated `fast_mod` that isolates remainder operations strictly without executing a full arbitrary-scale division stack.

Algorithm Pipeline: miller_rabin.cpp

We structure our test using a fixed witness array to allow deterministic reproducibility up to the native threshold before scaling constraints apply.

Algorithm Pipeline: miller_rabin.cpp

We structure our test using a fixed witness array to allow deterministic reproducibility up to the native threshold before scaling constraints apply.

- **Witness Set:** Fixed array $\{2, 3, 5, 7, 11, 13, 17, 19, 23\}$.
- **Prime Search Workflow:**
 - 1 Randomize bits between structural limits.
 - 2 Impose odd sizing by clamping LSB.
 - 3 *Pre-filtering:* Divide quickly by $2, 3, 5, \dots$ to filter out 90% of candidates cleanly.
 - 4 Pipe surviving candidates into full Miller-Rabin suite.

Algorithm Pipeline: miller_rabin.cpp

We structure our test using a fixed witness array to allow deterministic reproducibility up to the native threshold before scaling constraints apply.

- **Witness Set:** Fixed array $\{2, 3, 5, 7, 11, 13, 17, 19, 23\}$.
- **Prime Search Workflow:**
 - 1 Randomize bits between structural limits.
 - 2 Impose odd sizing by clamping LSB.
 - 3 *Pre-filtering:* Divide quickly by $2, 3, 5, \dots$ to filter out 90% of candidates cleanly.
 - 4 Pipe surviving candidates into full Miller-Rabin suite.

We also implemented sequential neighbor search targets, allowing the workflow to hunt for the `Next_Prime` and `Prev_Prime` off arbitrary starting data.

Interface and Visualization

The project encompasses fully featured endpoints so the C++ arithmetic isn't stranded.

Interface and Visualization

The project encompasses fully featured endpoints so the C++ arithmetic isn't stranded.

CLI Automation (`primality_backend.exe`)

- Takes parsed inputs and operates the C++ heavy-lifting behind the scenes.
- Output is strictly structured (Key-Value) to pipe into our benchmarking telemetry.

Interface and Visualization

The project encompasses fully featured endpoints so the C++ arithmetic isn't stranded.

CLI Automation (`primality_backend.exe`)

- Takes parsed inputs and operates the C++ heavy-lifting behind the scenes.
- Output is strictly structured (Key-Value) to pipe into our benchmarking telemetry.

GUI Interface (`primality_gui.py`)

- Full Tkinter interface wrapper allowing humans to input decimal & hexadecimal arrays, triggering background validations transparently.
- The same interface also exposes prime generation and neighbor-prime discovery, making the implementation demonstrable without touching the command line.

GUI Demonstration

Primality Checker

Input format: ☒ Decimal ☐ Hexadecimal

Enter number:

23421534645

Check Prime Clear

Generate probable prime with bit length: 128 Generate Prime

All methods agree. Neighbor search uses fast Miller-Rabin.

Number:
23421534645

Bit length: 35

Ordinary modulo Miller-Rabin:
Result: composite
Time: 0.001 ms

Fast modulo Miller-Rabin:
Result: composite
Time: 0.001 ms

AKS primality test:
Result: composite
Time: 0.518 ms

Neighbor search using fast Miller-Rabin:
Previous probable prime: 23421534571
Previous search time: 4.881 ms
Next probable prime: 23421534649
Next search time: 1.852 ms

Miller-Rabin agreement: yes
All methods agree: yes

Prime-checking workflow with timing and neighboring-prime output.

Primality Checker

Input format: ☒ Decimal ☐ Hexadecimal

Enter number:

10727558424076279817

Check Prime Clear

Generate probable prime with bit length: 64 Generate Prime

Probable prime generated by the C++ backend.

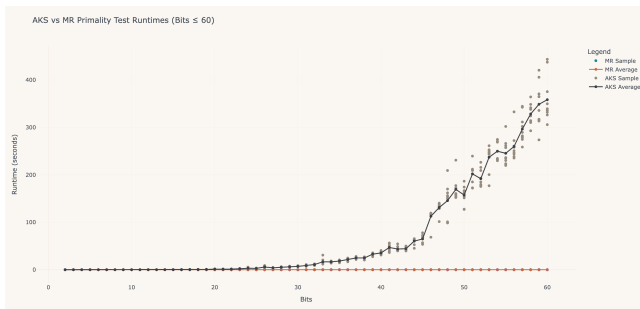
Generated probable prime:
10727558424076279817

Bit length: 64
Generation time: 24.438 ms

Prime-generation workflow for user-selected bit lengths.

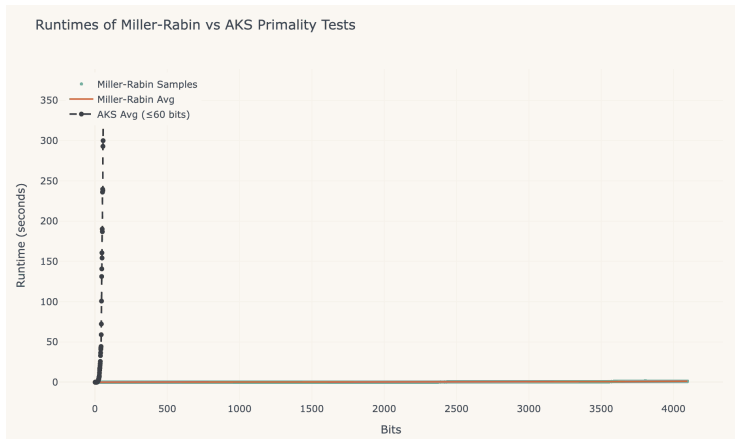
Observing Theory Vs Reality: The AKS Explosion

The goal of benchmarking is to observe the difference between deterministic polynomial bounds, and actual constant-factor engineering realities.



Practical Inference: While AKS is guaranteed polynomial $\mathcal{O}(\log^6 n)$, the graphical evidence shows a geometric wall. Testing a simple 60-bit prime severely delayed execution.

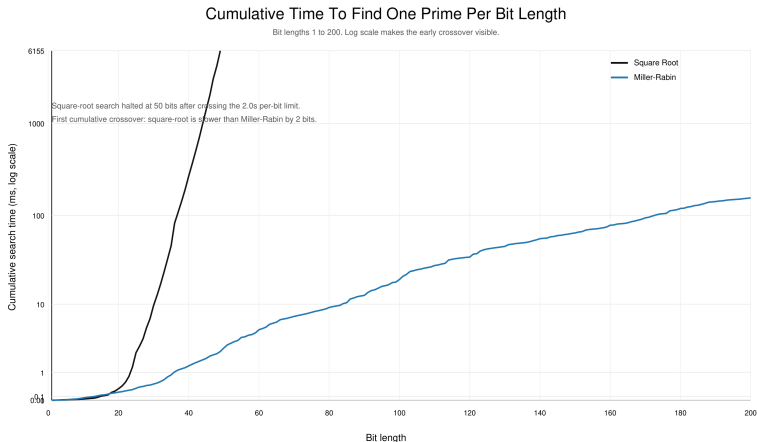
Observing Theory Vs Reality: Miller-Rabin Dominance



Practical Inference: Unlike AKS, Miller-Rabin operates on numbers in the thousands of bits while maintaining an execution profile almost visually flat against the base axis. For cryptographic needs (4096 bits), probabilistic testing is mandatory.

Log-Scale Runtime Distribution

Comparing Prime Searching algorithms across aggregated datasets:



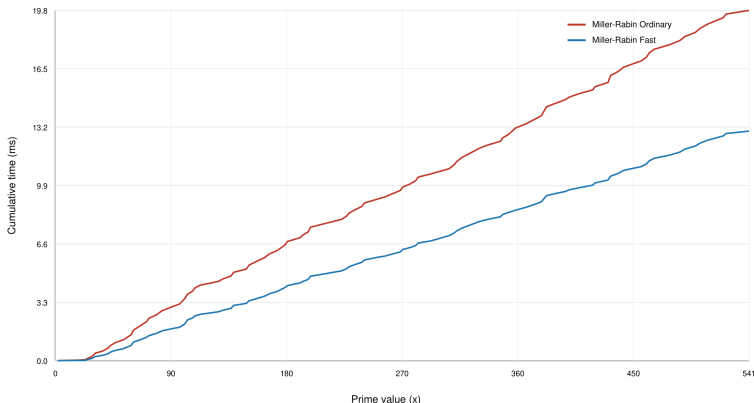
Practical Inference: The Square-root baseline hits a computational limit instantly by 50 bits. Miller-Rabin is strictly required for driving sweeping prime-search boundaries efficiently.

Engineering Isolation: Miller-Rabin Refinement

Even within a specific algorithm, custom engineering paths dictate performance.

Miller-Rabin Fast vs Ordinary

Direct cumulative comparison of the two Miller-Rabin modulo variants.



Practical Inference: Comparing standard modular divisions against our custom `fast_mod` engine highlights how critical low-level register isolation is. It saves vital overhead stacked exponentially across k -loops.

Conclusion

- **Mathematical Rigor:** We have observed step-by-step why the Miller-Rabin sequence rejects Carmichael combinations dynamically using field constraints.

Conclusion

- **Mathematical Rigor:** We have observed step-by-step why the Miller-Rabin sequence rejects Carmichael combinations dynamically using field constraints.
- **Theoretical Limits:** We use the standard result that for odd composite n , at most $\frac{\varphi(n)}{4}$ bases can be strong liars.

Conclusion

- **Mathematical Rigor:** We have observed step-by-step why the Miller-Rabin sequence rejects Carmichael combinations dynamically using field constraints.
- **Theoretical Limits:** We use the standard result that for odd composite n , at most $\frac{\varphi(n)}{4}$ bases can be strong liars.
- **Engineering Application:** Our custom 'big_int' and optimization filters bridged the gap between pure math and functional CPU execution cycles.

Conclusion

- **Mathematical Rigor:** We have observed step-by-step why the Miller-Rabin sequence rejects Carmichael combinations dynamically using field constraints.
- **Theoretical Limits:** We use the standard result that for odd composite n , at most $\frac{\varphi(n)}{4}$ bases can be strong liars.
- **Engineering Application:** Our custom 'big_int' and optimization filters bridged the gap between pure math and functional CPU execution cycles.
- **Final Takeaway:** A test that is "probabilistically true" executed in 150ms beats a test that is "deterministically true" but takes centuries. Engineering embraces manageable risk to unlock scale.

Conclusion

- **Mathematical Rigor:** We have observed step-by-step why the Miller-Rabin sequence rejects Carmichael combinations dynamically using field constraints.
- **Theoretical Limits:** We use the standard result that for odd composite n , at most $\frac{\varphi(n)}{4}$ bases can be strong liars.
- **Engineering Application:** Our custom 'big_int' and optimization filters bridged the gap between pure math and functional CPU execution cycles.
- **Final Takeaway:** A test that is "probabilistically true" executed in 150ms beats a test that is "deterministically true" but takes centuries. Engineering embraces manageable risk to unlock scale.

Thank You.

Selected References

- M. Agrawal, N. Kayal, and N. Saxena, *PRIMES is in P*, Annals of Mathematics, 160(2), 781–793, 2004.
- G. L. Miller, *Riemann's Hypothesis and Tests for Primality*, Journal of Computer and System Sciences, 1976.
- M. O. Rabin, *Probabilistic Algorithm for Testing Primality*, Journal of Number Theory, 1980.
- OpenAI, *ChatGPT*, used to write the AKS code implementation for this project, accessed April 5, 2026.